

Pair

pair

`std::pair` 是标准库中定义的一个类模板。用于将两个变量关联在一起，组成一个「对」，而且两个变量的数据类型可以是不同的。

► 类模板

通过灵活使用 `pair`，可以轻松应对 需要将关联数据捆绑存储、处理 的场景。

► Pair

使用

初始化

可以在定义时直接完成 `pair` 的初始化。

```
1 pair<int, double> p0(1, 2.0);
```

也可以使用先定义，后赋值的方法完成 `pair` 的初始化。

```
1 pair<int, double> p1;  
2 p1.first = 1;  
3 p1.second = 2.0;
```

还可以使用 `std::make_pair` 函数。该函数接受两个变量，并返回由这两个变量组成的 `pair`。

```
1 pair<int, double> p2 = make_pair(1, 2.0);
```

一种常用的方法是使用宏定义 `#define mp make_pair`，将有些冗长的 `make_pair` 化简为 `mp`。

在 C++11 以及之后的版本中，`make_pair` 可以配合 `auto` 使用，以避免显式声明数据类型。

```
1 auto p3 = make_pair(1, 2.0);
```

关于 `auto` 的在信息学竞赛中的使用，参见 [迭代器](#) 部分的说明。

访问

通过成员函数 `first` 与 `second`，可以访问 `pair` 中包含的两个变量。

```
1 int i = p0.first;  
2 double d = p0.second;
```

也可以对其进行修改。

```
1 p1.first++;
```

比较

`pair` 已经预先定义了所有的比较运算符，包括 `<`、`>`、`<=`、`>=`、`==`、`!=`。当然，这需要组成 `pair` 的两个变量所属的数据类型定义了 `==` 和/或 `<` 运算符。

其中，`<`、`>`、`<=`、`>=` 四个运算符会先比较两个 `pair` 中的第一个变量，在第一个变量相等的情况下再比较第二个变量。

```
1 if (p2 >= p3) {
2     cout << "do something here" << endl;
3 }
```

由于 `pair` 定义了 STL 中常用的 `<` 与 `==`，使得其能够很好的与其他 STL 函数或数据结构配合。比如，`pair` 可以作为 `priority_queue` 的数据类型。

```
1 priority_queue<pair<int, double>> q;
```

赋值与交换

可以将 `pair` 的值赋给另一个类型一致的 `pair`。

```
1 p0 = p1;
```

也可以使用 `swap` 函数交换 `pair` 的值。

```
1 swap(p0, p1);
2 p2.swap(p3);
```

应用举例

离散化

`pair` 可以轻松实现离散化。

我们可以创建一个 `pair` 数组，将原始数据的值作为每个 `pair` 第一个变量，将原始数据的位置作为第二个变量。在排序后，将原始数据值的排名（该值排序后所在的位置）赋给该值原本所在的位置即可。

```
1 // a为原始数据
2 pair<int, int> a[MAXN];
3 // ai为离散化后的数据
4 int ai[MAXN];
5 for (int i = 0; i < n; i++) {
6     // first为原始数据的值，second为原始数据的位置
7     scanf("%d", &a[i].first);
8     a[i].second = i;
9 }
10 // 排序
11 sort(a, a + n);
12 for (int i = 0; i < n; i++) {
13     // 将该值的排名赋给该值原本所在的位置
14     ai[a[i].second] = i;
15 }
```

Dijkstra

如前所述, `pair` 可以作为 `priority_queue` 的数据类型。

那么, 在 Dijkstra 算法的堆优化中, 可以使用 `pair` 与 `priority_queue` 维护节点, 将节点当前到起点的距离作为第一个变量, 将节点编号作为第二个变量。

```
1 priority_queue<pair<int, int>, std::vector<pair<int, int>>,
2             std::greater<pair<int, int>>>
3     q;
4 ... while (!q.empty()) {
5     // dis为入堆时节点到起点的距离, i为节点编号
6     int dis = q.top().first, i = q.top().second;
7     q.pop();
8     ...
9 }
```

pair 与 map

`map` 的是 C++ 中存储键值对的数据结构。很多情况下, `map` 中存储的键值对通过 `pair` 向外暴露。

```
1 map<int, double> m;
2 m.insert(make_pair(1, 2.0));
```

关于 `map` 更多的内容, 请见 [关联式容器](#) 与 [无序关联式容器](#) 中相关部分。

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [sbofgayschool](#), [CCXX XI](#), [Enter-tainer](#), [Tiphereth-A](#), [xizeyoupan](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用